

Malicious Activity & Detection

4/15/26 2:13 PM

Core Mindset

Goal: Detect malicious activity at different layers (file, network, logs). **Why:** Malware leaves traces in files, network traffic, and system logs. You combine all three for strong detection.

Three Detection Layers:

- **YARA:** "Does this file look like malware?" (static patterns)
- **Suricata:** "Is this network traffic suspicious?" (behavior in transit)
- **Sigma:** "Did this system behavior happen?" (logs / events)

Key Principle: Focus on **behavior patterns**, not just IOCs. Combine layers for better accuracy.

YARA – File Detection

What it is: Pattern-matching tool for files (and memory). **When to use:** Quick static analysis of suspicious files or binaries. **Why:** Fast way to find malware by matching strings, byte sequences, or code patterns.

Basic Rule Structure

```
yara
rule RuleName {
    meta:
        description = "Detects Azorult stealer"

    strings:
        $x1 = "cmd.exe /c timeout 3 & del" wide // required
        $x2 = "STEMPS\\curbuf.dat" wide
        $s1 = "SOFTWARE\\Microsoft\\Windows NT" wide // optional
        $s5 = "Software\\...\\Uninstall\\" base64

    condition:
        uint16(0) == 0x5A4D and // PE file (MZ header)
        all of ($x*) and
        4 of ($s*)
}
```

Key Concepts & Commands

- **Strings:** What to match on.
 - \$x* = required (highly unique)
 - \$s* = optional (contextual)
- **Modifiers:**
 - wide → UTF-16 (common in Windows malware)
 - nocase → case insensitive
 - base64 → match encoded version
 - ascii → normal text
- **Condition:**
 - uint16(0) == 0x5A4D → PE file check
 - all of (\$x*) → must match all required
 - 4 of (\$s*) → flexible matching

Command:

Bash

```
yara rule.yar sample.exe
```

When to use hex patterns instead of strings:

- No readable strings (packed/obfuscated)
- Detect packers (e.g., PECompact2)
- Use IDA Pro to find unique code sequences at entry point

Main Takeaway: YARA = fingerprint files by content. Balance unique + contextual strings.

Suricata – Network Detection

What it is: IDS/IPS for network traffic. **When to use:** Analyze PCAPs or live traffic for C2, downloads, exfiltration. **Why:** Reveals what malware does over the network (even if file is obfuscated).

Basic Rule Structure

```
suricata

alert http $HOME_NET any -> $EXTERNAL_NET any (
    msg:"Amadey C2 Check-in";
    flow:established,to_server;
    http.method; content:"POST";
    http.uri; content:"index.php"; endswith;
    http.header_names; content:! "Referer";
    http.request_body; content:"st="; offset:0; depth:3;
    sid:1000;
)
```

Key Concepts & Commands

- **Action:** alert (generate alert), drop (block)
- **Header:** protocol, IPs, ports, direction (->)
 - \$HOME_NET = internal
 - \$EXTERNAL_NET = internet
- **Sticky Buffers** (modern way):
 - http.method; content:"POST";
 - http.uri;
 - http.request_body;
 - tls.sni;
- **Modifiers:**
 - nocase
 - startswith / endswith
 - offset / depth (absolute)
 - distance / within (relative)
- **Flow:**
 - flow:established,to_server; → client to server only

TLS Detection (encrypted traffic):

- Use metadata:
 - tls.sni → domain
 - ja3 / ja3s → fingerprints
 - tls.cert_issuer → self-signed certs

Flowbits (correlate rules):

- Rule 1: flowbits:set,c2_active; noalert;
- Rule 2: flowbits:isset,c2_active;

Command:

Bash

```
suricata -S custom.rules -r sample.pcap
```

Main Takeaway: Suricata = detect behavior in network traffic. Use sticky buffers + flow for precision.

Sigma – Log Detection

What it is: Universal rule format for logs (write once, convert anywhere). **When to use:** Detect suspicious system behavior in Windows events / SIEM. **Why:** Vendor-agnostic. Convert to Splunk, Elastic, PowerShell, etc.

Basic Rule Structure

```
YAML

title: Suspicious Office Spawning PowerShell
status: experimental

logsource:
  category: process_creation
  product: windows

detection:
  selection:
    ParentImage|endswith: winword.exe
    Image|endswith: powershell.exe
    CommandLine|contains|all:
      - "-e"
      - "hidden"
      - "-nop"

condition: selection
```

Key Concepts

- **logsource:** Defines data (category, product, service)
- **detection:**
 - selection: your logic (maps or lists)
 - Modifiers: contains, endswith, startswith, all
- **condition:** condition: selection → "alert if selection matches"

Conversion (MUST DO):

Bash

```
sigma plugin install powershell
sigma convert --target powershell rule.yml
```

→ Turns Sigma into real query (PowerShell, Splunk SPL, Elastic KQL)

Common Patterns:

- Process creation: Parent = winword.exe + Child = powershell.exe + encoded args
- Persistence: schtasks.exe /Create /TR from Temp
- Ransomware: file creation ending in .README.txt in Desktop/Downloads

Main Takeaway: Sigma = log behavior detection. Write once → convert to any SIEM.

How the Three Layers Work Together

Attack Chain Example (Malicious Document):

1. **YARA:** Detects malicious Word doc or embedded payload (strings, MZ header).
2. **Suricata:** Detects network activity (PowerShell download, C2 POST, DLL fetch).
3. **Sigma:** Detects system behavior (Word → PowerShell with -e hidden -nop, schtasks, ransom note).

Why combine them:

- YARA catches the file
- Suricata catches the network
- Sigma catches the host impact
- Together = full visibility + fewer false positives

Detection Mindset:

- Start with behavior (what malware does)
- Choose the right layer
- Build precise patterns
- Test & tune (goodware vs malware)

Overall Detection Workflow

1. **Analyze sample** — sandbox (behavior), strings, PCAP, logs.
2. **Identify patterns** — unique strings, commands, traffic, file names.
3. **Choose layer:**
 - File → YARA
 - Network → Suricata
 - Logs → Sigma
4. **Write rule** — use required + contextual indicators.
5. **Test:**
 - YARA: yara rule.yar file
 - Suricata: suricata -S rules -r pcap
 - Sigma: convert → run query
6. **Tune** — reduce false positives, improve specificity.
7. **Document** — IOCs, family, behavior.

Final Takeaways:

- YARA = file fingerprints
- Suricata = network behavior (use sticky buffers + flow)
- Sigma = log behavior (convert before use)
- Best detection = combine all three layers
- Focus on behavior chains, not single IOCs
- Always test rules (goodware + malware)